SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*,

CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI

problems efficiently. (BL3)

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I, S.SUSITHRA(1912260033) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a

flood-affected region. The environment is highly dynamic—roads may suddenly become

inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic

estimate (straight-line distance), Partial real-time updates about blocked paths and Variable

movement costs depending on terrain and weather. However, due to urgency, the drone must

quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its

strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and

heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify

ANSWER:

AI-Powered Drones for Emergency Medicine Delivery

i. Greedy Best-First Search: Behaviour, Strengths and Weaknesses

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the next node based only on the heuristic value $h(n)$.

$$ f(n)=h(n) $$

Here, $h(n)$ represents the estimated straight-line distance from the drone's current position to the medicine delivery destination.

Working:

1. The drone identifies its current location and destination.
2. It estimates the distance from each available location to the destination.
3. It selects the location with the smallest heuristic value.

4. It continues until it reaches the destination or encounters a blocked path.

Strengths:

- Fast decision-making because it focuses on the destination.
- Requires less computation than many cost-based search methods.
- Useful when medicines must be delivered urgently.
- Can work with partial information about the environment.

Weaknesses:

- May select a short route that has dangerous weather or difficult terrain.
- Does not consider the total cost already incurred.
- May choose a blocked route when updates are incomplete.
- Does not guarantee the shortest or safest route.

ii. A* Search and Its Balance Between Cost and Heuristic

A* is an informed search algorithm that considers both the actual cost of reaching a node and the estimated cost to reach the destination.

$$[f(n)=g(n)+h(n)]$$

Where:

- \(g(n)\): Actual cost from the starting point to node \(n\).
- \(h(n)\): Estimated cost from node \(n\) to the destination.
- \(f(n)\): Estimated total route cost.

Working:

1. The drone calculates the cost of travelling through available paths.
2. It estimates the remaining distance to the destination.
3. It selects the node with the lowest \(f(n)\).
4. It updates route costs when new information becomes available.
5. If a path becomes blocked, it replans using the updated information.

For example, a short route through a storm may have a high total cost, while a slightly longer route through clear weather may be safer and faster overall.

Advantages: A* considers route costs, supports informed decision-making and can find an optimal route when its assumptions are satisfied.

Limitation: It may require significant memory and computation, especially when the environment changes frequently.

iii. Impact of Heuristic Accuracy on Both Algorithms

A heuristic estimates the remaining cost to reach the goal.

- Accurate heuristic: Helps identify promising routes quickly.
- Overestimated heuristic: May lead A* to lose its optimality guarantee.
- Underestimated heuristic: If admissible, it helps A* retain optimality, although it may explore more nodes.
- Poor heuristic: Can cause unnecessary exploration or selection of an unsuitable route.

In GBFS, the heuristic strongly determines the selected path because the algorithm ignores the cost already travelled. In A*, the heuristic is combined with the actual path cost, so a poor heuristic generally affects efficiency and may affect optimality if its required conditions are violated.

iv. Effect of Dynamic Changes and Blocked Paths

Floods and changing weather can make previously available routes unusable.

- The drone must receive updated information from sensors or communication systems.
- Blocked routes must be removed or marked unavailable.
- GBFS may need to restart or recalculate its search when the selected route becomes blocked.
- A* must update affected path costs and search for an alternative route.
- Repeated replanning increases processing time and can delay medicine delivery.

Incremental algorithms such as D Lite* can reuse previous search information to replan efficiently when the environment changes.

v. Recommended Algorithm and Justification

Recommendation: A with dynamic replanning, or D Lite for frequent route changes.

Factor	**GBFS**	**A***
Speed	Usually, fast	May require more computation
Path cost	Ignores travelled cost	Considers actual and estimated cost
Optimality	Not guaranteed	Guaranteed under suitable conditions
Dynamic changes	Can recalculate	Can update or replan
Safety	Needs extra risk checks	Can include risk in route cost

A* is suitable when route quality, delay and risk matter. The route cost can combine travel time, weather risk, battery usage and hazard penalties. If rapid changes occur frequently, D* Lite can reduce repeated search work.

Conclusion: A* with real-time updates and safety constraints is a strong choice because it balances efficiency and route quality. However, the drone should also have emergency landing rules and a safe fallback route.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However, the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

ANSWER:

Smart City Traffic Signal Optimization

Formulation as an Optimization Problem

The objective is to determine signal timings that minimize total waiting time, fuel consumption and congestion across multiple intersections.

Let the objective function be:

$$\min Z=w_1W+w_2F+w_3C$$

Where:

- W: Total vehicle waiting time.
- F: Total fuel consumption.
- C: Traffic congestion measure.
- w_1,w_2,w_3: Weights representing the importance of each objective.

Constraints:

- Minimum and maximum green-light durations.
- Safe pedestrian crossing intervals.
- No conflicting traffic movements receiving green simultaneously.

- Coordination between nearby intersections.
- Emergency vehicle priority.
- Traffic signals must satisfy applicable safety rules.

Traditional exhaustive search is inefficient because the number of possible timing combinations grows rapidly with the number of intersections and signal phases. Traffic conditions also change continuously, making repeated exhaustive searches impractical.

i. Working of Hill Climbing

Hill Climbing is a local search algorithm that repeatedly moves from the current solution to a neighbouring solution with a better objective value.

Working:

1. Generate an initial set of traffic signal timings.
2. Calculate waiting time, fuel consumption and congestion.
3. Make a small change to one or more green-light durations.
4. Evaluate the new timing plan.
5. Accept the change if it improves the objective.
6. Repeat until no better neighbouring solution is found.

For example, if vehicles are waiting longer at one intersection, the algorithm may increase its green-light duration and evaluate the effect on nearby intersections.

Advantages:

- Simple to implement.
- Uses relatively little memory.
- Can improve a solution quickly.
- Suitable for local adjustments in real time.

Limitations:

- May get trapped in a local optimum.
- Can struggle with large, complex search spaces.
- A change that helps one intersection may worsen congestion elsewhere.
- Does not guarantee a globally optimal solution.

ii. Local Maxima, Plateaus and Ridges

In a minimization problem, these difficulties can be understood using the equivalent idea of local minima and flat or narrow regions of the objective landscape.

1. Local optimum: A solution is better than all its immediate neighbours but is not the best overall. For example, improving one junction may not help because neighbouring junctions also need coordinated changes.

2. Plateau: Many neighbouring solutions have nearly equal performance. The algorithm cannot identify a clear direction for improvement, such as when small timing changes do not noticeably reduce waiting time.

3. Ridge: The best direction requires coordinated changes across multiple variables. For example, two adjacent intersections may need their green phases adjusted together to create a smooth traffic flow. Changing only one signal may not improve the result.

These conditions can cause Hill Climbing to stop before finding an effective city-wide timing plan.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) accepts better solutions and sometimes accepts worse solutions to escape local optima.

Its acceptance probability for a worse solution is:

\[P=e^{-\Delta E/T} \]

Where \(\Delta E\) is the increase in objective cost and \(T\) is the temperature parameter.

- At high temperatures, the algorithm explores more alternatives.
- As temperature decreases, it becomes more selective.
- This helps it escape local optima early and refine solutions later.

Genetic Algorithm (GA) uses principles inspired by natural selection.

1. Generate a population of possible signal-timing plans.
2. Evaluate each plan using the objective function.
3. Select better-performing plans.
4. Apply crossover to combine parts of different plans.
5. Apply mutation to introduce new timing combinations.
6. Repeat until a stopping condition is met.

GA explores several candidate solutions simultaneously and can handle complex interactions between intersections. However, it requires suitable parameter tuning and may be computationally expensive.

iv. Recommended Approach

Recommendation: A hybrid Genetic Algorithm with local search and real-time traffic feedback.

Factor	Hill Climbing	Simulated Annealing	Genetic Algorithm
Search style	One current solution	One solution with probabilistic exploration	Population of solutions

Escapes local optima	Poorly	Yes, probabilistically	Often, through diversity
Scalability	Good for simple local changes	Depends on cooling schedule	Can be parallelized
Adaptability	Fast local updates	Flexible exploration	Can optimize coordinated timings
Main limitation	Local optimum	May take many iterations	Population evaluation cost

A practical system should use a GA to optimize coordinated timings, local search to refine promising solutions, and live sensor data to update the plan. Computation should be restricted to a fixed time budget, with safe fallback signal timings always available.

Conclusion: A hybrid GA and local-search approach offers a useful balance between global exploration, local refinement and adaptation to changing traffic conditions.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It

must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited

sensing capability and make decisions without a complete map. The rover updates its

knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

ANSWER:

Autonomous Mars Rover Using Online Search

i. Why an Online Search Agent Is Required

An online search agent makes decisions while interacting with the environment. It does not need a complete map before starting its journey.

An offline search agent plans using a known environment and available information before taking action. However, the Mars rover operates in an unfamiliar environment where the complete terrain is unknown.

Why online search is required:

1. The rover cannot initially identify every crater, rock or obstacle.
2. Its sensors provide information only about nearby surroundings.
3. It must move, observe and update its knowledge continuously.
4. New obstacles may require changes to the planned route.
5. It must make decisions with limited communication and processing resources.

For example, when the rover discovers a crater that was not shown on its initial map, it must identify an alternative route instead of continuing along the blocked path.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability: The rover cannot observe the entire environment at once. Sensors provide incomplete or uncertain information.

Challenges include:

- Hidden obstacles and unknown terrain.
- Limited sensor range and measurement errors.
- Uncertainty about surface stability.
- Difficulty estimating the safest path.

Dynamic environment: The rover may encounter changing environmental conditions, shifting surface materials or unexpected equipment limitations.

These conditions require the rover to update its decisions continuously. Since communication with Earth may involve significant delays, it cannot depend on instructions from ground control for every movement.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an internal representation of its surroundings, including known terrain, obstacles, traversable regions and possible routes.

Working process:

1. Sense: Collect data using cameras, distance sensors and other instruments.
2. Interpret: Identify obstacles, craters and safe terrain.
3. Map: Build or update a local terrain map.
4. Plan: Select a promising path toward the target.
5. Act: Move a limited distance along the selected path.
6. Verify: Compare the expected result with the actual observations.
7. Replan: Modify the route if new hazards or unexpected conditions appear.

This process forms a continuous cycle of perception, planning, action and learning.

iv. Comparison of Online, Uninformed and Informed Search

Feature	Online Search	Uninformed Search	Informed Search
Environment knowledge	Learns while acting	Uses the given problem description	Uses problem information and a heuristic
Heuristic	Not always required	Not required	Usually required
Adaptability	High	Depends on replanning	Depends on algorithm and updates
Examples	Online agent, online DFS	BFS, DFS, UCS	A*, Greedy Best-First
Main limitation	Decisions under uncertainty	May explore many nodes	Depends on heuristic quality

Online search describes how an agent explores and acts without complete prior knowledge. Uninformed and informed search describe how a search algorithm selects and explores states. These categories are not mutually exclusive: an online rover can use heuristic information when it becomes available.

v. Most Suitable Approach

Recommendation: Online heuristic search with a local map, A-based planning and continuous safety checks.*

Reasons:

- Uncertainty: The rover learns about unknown terrain through sensor observations.
- Adaptability: It can update the map and replan after discovering obstacles.
- Efficiency: Heuristics help prioritize routes toward sample locations.
- Safety: Hazard costs can penalize routes close to craters or unstable ground.
- Autonomy: The rover can operate even when immediate communication with Earth is unavailable.

A risk-aware planner should also enforce limits on slope, obstacle clearance, battery usage and movement speed. If uncertainty becomes too high, the rover should stop or move to a verified safe location.

Conclusion: An online search agent is most suitable because it combines real-time action with continuous learning and route adaptation in an incompletely known environment.

4. Scenario: A university is designing an AI system to automatically generate exam

timetables. The system must satisfy multiple constraints: No student should have overlapping

exams, Limited number of exam halls, Certain subjects must be scheduled before others,

Faculty availability constraints, Special accommodations for some students, The complexity

increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

ANSWER:

University Exam Timetable Generation Using CSP

Formulation as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem (CSP) consists of three main components:

$$ CSP=(X,D,C) $$

Where:

- X: Set of variables.
- D: Domains of possible values for the variables.
- C: Constraints that restrict valid assignments.

The objective is to assign an examination time and hall to every course while satisfying all hard constraints.

i. Variables, Domains and Constraints

1. Variables

For each course i, define:

- T_i: Examination time slot.
- H_i: Examination hall.

For example, T_{Maths} represents the slot assigned to the Mathematics examination.

2. Domains

A domain is the set of possible values for a variable.

- $D(T_i)$: Available examination slots.
- $D(H_i)$: Available examination halls.

For example:

$$ D(T_{\text{Maths}})=\{S_1,S_2,S_3\} $$

If three halls are available:

\[D(H_i)=\{H_1,H_2,H_3\} \]

Domains can exclude slots when a faculty member is unavailable or when a student requires a particular accommodation.

3. Constraints

- No overlapping exams: A student must not have two examinations in the same time slot.
- Hall availability: A hall cannot host two examinations simultaneously.
- Hall capacity: The hall must accommodate all students assigned to that examination.
- Precedence: If Course A must occur before Course B, then \(T_A<T_B\).
- Faculty availability: Examinations requiring a particular faculty member must respect their availability.
- Special accommodations: Accessible halls and suitable time arrangements must be provided where required.
- Resource limitations: The timetable must use only available slots and halls.

Hard constraints must be satisfied. Preferences such as avoiding late examinations can be treated as soft constraints and optimized after obtaining a valid timetable.

ii. How Backtracking Search Works

Backtracking is a depth-first search technique that assigns values to variables one at a time.

Steps:

1. Select an unassigned course.
2. Choose an available time slot.
3. Assign a suitable examination hall.
4. Check whether the assignment violates any constraint.
5. If it is valid, proceed to the next course.
6. If a conflict occurs, undo the assignment and try another value.
7. Continue until a complete valid timetable is generated.

Example:

Suppose Mathematics and Physics share some students. If Mathematics is scheduled in Slot 1, Physics cannot be scheduled in Slot 1. The algorithm tries another slot for Physics. If no valid slot remains, it backtracks and changes an earlier assignment.

This method systematically explores possible assignments without retaining invalid partial solutions.

iii. Constraint Propagation and Forward Checking

Constraint propagation reduces the domains of unassigned variables whenever a new assignment is made.

Forward checking works as follows:

1. Assign Mathematics to Slot 1.
2. Remove Slot 1 from the domains of examinations that share students with Mathematics.
3. Remove unavailable halls from relevant hall domains.
4. Check whether any unassigned variable has an empty domain.
5. If a domain becomes empty, backtrack immediately.

For example, if Physics has only Slots 1 and 2 available, and Mathematics is assigned Slot 1, forward checking reduces the Physics domain to Slot 2. This prevents the algorithm from exploring assignments that are already known to be invalid.

Other useful techniques include:

- Minimum Remaining Values (MRV): Select the variable with the fewest legal values.
- Degree heuristic: Prioritize the variable involved in the largest number of constraints.
- Least Constraining Value: Choose the value that eliminates the fewest choices for other variables.

These methods reduce unnecessary search and help solve larger timetabling problems.

iv. Real-World Challenges and Best Strategy for Scalability

Real-world challenges include:

- Thousands of students and many courses.
- Limited halls and examination slots.
- Faculty schedule changes.
- Students taking multiple courses.
- Accessibility and special accommodation requirements.
- New courses or last-minute cancellations.

A simple backtracking algorithm may become slow as the number of variables and constraints increases.

Recommended strategy: CSP with constraint propagation, MRV, degree heuristic and backtracking.

For very large problems, a hybrid approach can use a CSP solver for hard constraints and local search or optimization methods for soft preferences.

Reasons:

- Constraint propagation eliminates impossible assignments early.

- MRV prioritizes the most difficult courses.
- Backtracking guarantees a systematic search for a solution.
- Optimization methods can improve convenience and reduce timetable gaps.
- Incremental rescheduling can preserve unaffected examinations when changes occur.

Conclusion: A CSP-based system with intelligent variable ordering and constraint propagation is suitable for generating valid and scalable examination timetables.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta

Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The

AI must complete against human players, Make decisions within strict time limits, Evaluate

complex game states with multiple possible moves, The decision tree is extremely large,

making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

ANSWER:

Strategic Game AI Using Minimax and Alpha-Beta Pruning

i. How Minimax Models the Problem

Minimax is a decision-making algorithm used in competitive, adversarial games. It assumes that both players choose moves to maximize their own advantage.

The game is represented as a tree:

- MAX player: Attempts to maximize the evaluation score.
- MIN player: Attempts to minimize the evaluation score.
- Nodes: Represent game states.
- Edges: Represent possible moves.
- Leaf nodes: Represent terminal states or states evaluated at a search-depth limit.

The minimax rule is:

$$ V(n)= \begin{cases} \max V(c), & \text{MAX node}\\ \min V(c), & \text{MIN node} \end{cases} $$

Here, c represents a child state of node n.

Working:

1. Generate possible moves from the current state.
2. Expand the game tree to a selected depth.
3. Evaluate the leaf states.
4. Propagate scores upward through MAX and MIN nodes.
5. Select the move with the highest minimax value.

For example, if the AI considers three moves, it evaluates the opponent's strongest responses to each move and chooses the option with the best guaranteed outcome.

ii. Computational Challenges of Large Game Trees

The main problem with Minimax is its high computational cost.

If the average branching factor is b and the search depth is d, the number of nodes examined is approximately:

$$ O(b^d) $$

For example, if each state has 10 possible moves and the algorithm searches 5 levels, the tree may contain around 10^5 leaf paths.

Major challenges include:

- Large branching factor: Many possible moves at each state.
- Deep search: Tactical consequences may occur several moves later.
- Time limits: The AI must respond before the allotted decision time expires.
- Memory usage: Storing states and move information can consume substantial resources.
- Uncertainty: Opponents may behave unpredictably, and some games contain hidden information.

As a result, a full-depth Minimax search is often impractical in complex strategy games.

iii. Alpha-Beta Pruning and Detailed Reasoning

Alpha-Beta pruning improves Minimax by eliminating branches that cannot affect the final decision.

It maintains two values:

- α: The best value MAX can guarantee so far.
- β: The best value MIN can guarantee so far.

Pruning occurs when:

$$ \alpha \geq \beta $$

Example:

Suppose MAX has two possible moves, A and B.

1. Move A has an evaluated value of 5, so MAX can guarantee at least 5.
2. While evaluating B, MIN discovers a response with a value of 3.
3. MIN can choose that response, so B cannot achieve a value greater than 3 under optimal play.
4. Since B cannot beat A's value of 5, the remaining branches under B can be skipped.

The algorithm therefore avoids evaluating branches that cannot change the best move.

Advantages:

- Produces the same result as Minimax when both use the same search depth and evaluation assumptions.
- Reduces the number of states evaluated.
- Allows deeper searches within the same time budget.
- Becomes more efficient when promising moves are examined first.

In the best case, good move ordering can reduce the effective time complexity toward $O(b^{d/2})$. In the worst case, its complexity remains $O(b^d)$.

iv. Design of an Evaluation Function

An evaluation function estimates the quality of a non-terminal game state when the search reaches its depth limit.

A possible function for a strategy game is:

$$E(s)=w_1M+w_2R+w_3T+w_4P+w_5O$$

Where:

- M: Military strength and unit advantage.
- R: Resource availability.
- T: Control of important territory.
- P: Progress toward the victory objective.
- O: Operational position, such as defensive strength and unit safety.
- $w_1,\ldots,w_5$: Weights representing the importance of each factor.

Justification:

- Military strength helps estimate the ability to defeat the opponent.
- Resources determine whether the AI can build units and sustain operations.
- Territory control provides strategic and positional advantages.
- Victory progress rewards actions that directly support the game objective.
- Defensive position reduces the likelihood of losing valuable units.

The factors should be normalized where appropriate, and their weights should be tuned to the game's rules. Terminal wins and losses should receive scores much larger in magnitude than ordinary positional evaluations.

v. Recommended Strategy for Optimality and Real-Time Performance

Recommendation: Iterative deepening Minimax with Alpha-Beta pruning, move ordering and a time limit.

A practical system should:

1. Search to depth 1, then depth 2, depth 3 and so on.
2. Order promising moves first to improve pruning.
3. Stop before the decision deadline.
4. Retain the best move from the last completed search.
5. Use a carefully designed evaluation function at non-terminal leaf states.
6. Apply a transposition table to reuse results for previously evaluated states.

Iterative deepening is particularly useful because the AI always has a completed, valid search result available if the time limit is reached.

Trade-off: Minimax with Alpha-Beta pruning can be optimal for a fully searched finite game tree under its assumptions. When the search is cut off at a limited depth, however, the chosen move depends on the accuracy of the evaluation function.